
初期粒子の発生方法

演習パッケージ: P07_PrimaryGenerator

Geant4 11.3準拠

Geant4講習会資料: HEP/NP/Space 2025



大学共同利用機関法人
高エネルギー加速器研究機構

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - ・ Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - ・ 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません

演習トピックス

[概要] P07_PrimaryGeneratorプログラムを使い、初期粒子発生の設定方法を学ぶ

[目次]

1. P07_PrimaryGenerator/Gun
 - Particle Gunの設定方法
2. P07_PrimaryGenerator/GPS
 - General Particle Sourceの使い方
3. P07_PrimaryGenerator/MyGen
 - 独自の初期粒子発生方法を学ぶ
4. P07_PrimaryGenerator/MyGen_MagField
 - 一様磁場の設定方法を学ぶ

P07_PrimaryGenerator/Gun

Particle Gunの設定方法

演習トピックス

[概要] [P07_PrimaryGenerator](#)プログラムを使い、初期粒子発生の設定方法を学ぶ

[目次]



1. [P07_PrimaryGenerator/Gun](#)
 - Particle Gunの設定方法
2. [P07_PrimaryGenerator/GPS](#)
 - General Particle Sourceの使い方
3. [P07_PrimaryGenerator/MyGen](#)
 - 独自の初期粒子発生方法を学ぶ
4. [P07_PrimaryGenerator/MyGen_MagField](#)
 - 一様磁場の設定方法を学ぶ

P07_PrimaryGenerator/Gun プログラムの概要

■ 演習プログラムの目的

- 今までの演習で使ってきたG4ParticleGunがどのように設定されているかを学ぶ

■ プログラムの構成

- mainプログラム: P06_Geometry/Pixel_Oneと同一
- ジオメトリ: P06_Geometry/Pixel_Oneと同一
- PhysicsList: P06_Geometry/Pixel_Oneと同一
- PrimaryGenerator: P06_Geometry/Pixel_Oneと同一

■ 組み込まれているジオメトリと粒子発生機能

- 初期粒子はマクロ(primaryGeneratorSetup.mac)で設定

照射粒子:

陽子

50 MeV/c

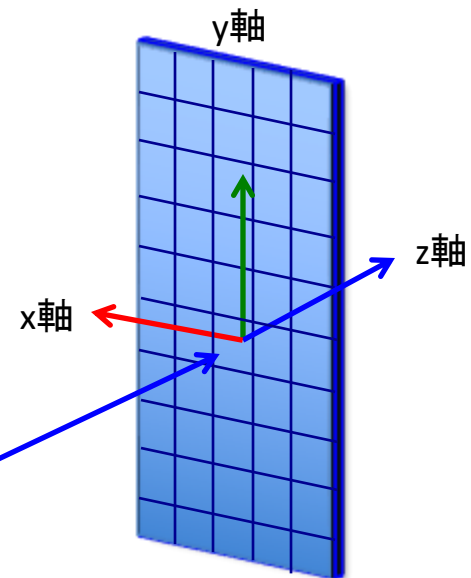
照射原点:

$x = 0,$

$y = 0,$

$z = -5.0 \text{ cm}$

G4ParticleGunによる点源



P07_PrimaryGenerator/Gunのビルド

[注] コマンド入力は「tabキー」補完機能を使う

課題:1 プログラム構成を理解する

- 前演習(P06_Geometry/Pixel_One)の全コードがP07_PrimaryGenerator/Gunにコピーされている

課題:2 P07_PrimaryGenerator/Gunのソースコードをビルドして実行する

1) P07_PrimaryGenerator/Gunのディレクトリに移行する

```
$ cd ~/Geant4Tutorial_2025/P07_PrimaryGenerator/Gun
$ ls
source/  util/  TestBench/
```

- sourceはP06_Geometry/Pixel_Oneと完全同一
- TestBenchのPrimaryGeneratorSetup.macには50MevのProtonが設定されている

2) ソースをビルド

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

← Scriptを走らせるとアプリ実行環境が完成

← ソースのビルドでbin、buildが作られた

3) TestBench(作業ディレクトリ)に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

P07_PrimaryGenerator/Gunプログラムを使う

課題:1 実行されたプログラムを使う

1) Qt画面のコマンド窓に以下を入力しアプリ実行

```
/control/execute GlobalSetup.mac
```

または、Qtのファイルアイコンをクリック

2) Qtコマンド窓に以下のコマンドを入力

```
/run/beamOn 1
```

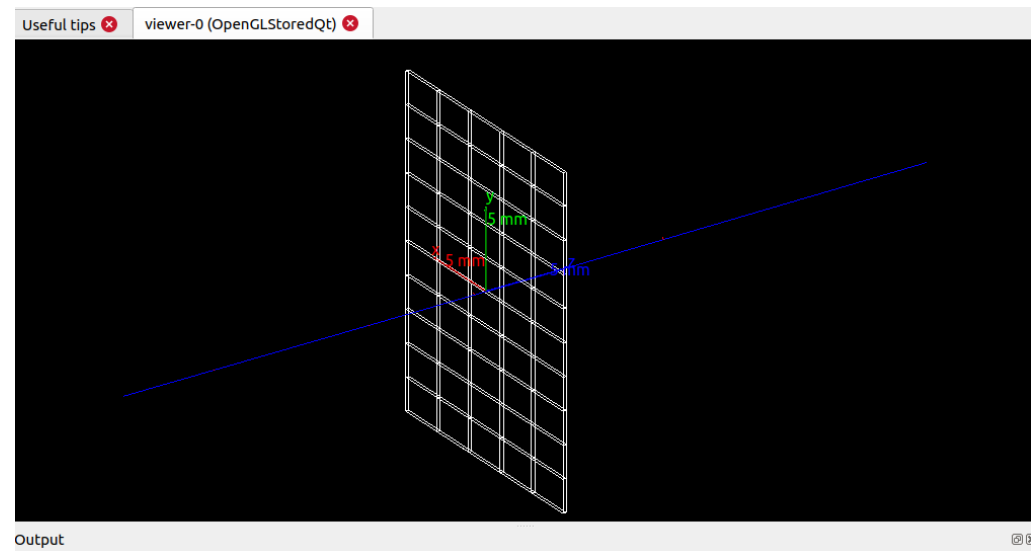
- 右図の様にprotonの飛跡が見えることを確認

[注] TestBenchのprimaryGeneratorSetup.macに
初期粒子としてproton (50 Mev)が設定

3) アプリを終了

```
exit
```

次の課題では、実行したプログラムの
PrimaryGeneratorクラスのコードを読み、
実装手法を学ぶ



P07_PrimaryGenerator/Gun: PrimaryGenerator.hhの内容

課題:1 初期粒子定義ヘッダーファイルを理解する

```
$ less ../source/include/PrimaryGenerator.hh
```

```
//+++++
// PrimaryGenerator.hh
//+++++
#ifndef PrimaryGenerator_h
#define PrimaryGenerator_h 1

#include "G4VUserPrimaryGeneratorAction.hh"
class G4Event;
class G4ParticleGun;

//-----
class PrimaryGenerator: public G4VUserPrimaryGeneratorAction
//-----
{
public:
    PrimaryGenerator();
    ~PrimaryGenerator() override;

public:
    void GeneratePrimaries( G4Event* ) override;

private:
    G4ParticleGun* fpParticleGun;
};
```

ユーザ定義のクラス名は任意であるが、Hands-onでは、この名前を統一的に使う

ユーザの初期粒子定義は必ずこの仮想クラスを継承して行う

初期粒子発生の実装はこの関数の中で行う

P07_PrimaryGenerator/Gun: PrimaryGenerator.ccの内容

課題:2 初期粒子定義実装ファイルを理解する

```
$ less ../source/src/PrimaryGenerator.cc
```

```
//+++++
// PrimaryGenerator.cc
//+++++
#include "PrimaryGenerator.hh"
#include "G4ParticleGun.hh"

//-----
PrimaryGenerator::PrimaryGenerator()
: fpParticleGun{nullptr}
//-----
{
    fpParticleGun = new G4ParticleGun{};
}

//-----
PrimaryGenerator::~~PrimaryGenerator()
//-----
{
    delete fpParticleGun;
}

//-----
void PrimaryGenerator::GeneratePrimaries( G4Event* anEvent )
//-----
{
    fpParticleGun->GeneratePrimaryVertex( anEvent );
}
```

G4ParticleGunヘッダー
ファイルのインクルード

ParticleGunオブジェクト
の生成

ParticleGunを使って
初期粒子を発生

初期粒子の設定

課題 3: PrimaryGeneratorの設定方法を理解する

- Primary Generator設定は以下の2ステップで行う

Step 1: まず、UserActionInitializationの中にPrimaryGeneratorを登録

← Primary Generator設定はUser Actionsの一つ

Step 2: 上のUserActionInitializationをRunManagerへ登録

UserActionInitialization.cc

```
//+++++
// UserActionInitialization.cc
//+++++
#include "UserActionInitialization.hh"
#include "PrimaryGenerator.hh"

//-----
UserActionInitialization::UserActionInitialization()
: G4VUserActionInitialization()
//-----
{}

//-----
UserActionInitialization::~UserActionInitialization()
//-----
{}

//-----
void UserActionInitialization::Build() const
//-----
{
    SetUserAction{ new PrimaryGenerator{} };
}
```

Step 1

Application_Main.cc

```
//+++++
// Geant4 Tutorial for Hep/Space/Medicine Users
//+++++
#include "Geometry.hh"
#include "UserActionInitialization.hh"

#include "G4RunManagerFactory.hh"
#include "G4UImanager.hh"
#include "G4VisExecutive.hh"
#include "G4UIExecutive.hh"
#include "FTFP_BERT.hh"

//-----
int main( int argc, char** argv )
//-----
{
    // Construct a run manager whose type is specified by the environment
    auto* runManager = G4RunManagerFactory::CreateRunManager();

    // Set up mandatory user initialization: Geometry
    runManager->SetUserInitialization( new Geometry{} );

    // Set up mandatory user initialization: Physics-List
    runManager->SetUserInitialization( new FTFP_BERT{} );

    // Set up user initialization: User Actions
    runManager->SetUserInitialization( new UserActionInitialization{} );

    // Initialize G4 kernel
    runManager->Initialize();

    (以下省略)
```

Step 2

P07_PrimaryGenerator/GPS

General Particle Sourceの使い方

演習トピックス

[概要] [P07_PrimaryGenerator](#)プログラムを使い、初期粒子発生の設定方法を学ぶ

[目次]

1. [P07_PrimaryGenerator/Gun](#)
 - Particle Gunの設定方法
2. [P07_PrimaryGenerator/GPS](#)
 - General Particle Sourceの使い方
3. [P07_PrimaryGenerator/MyGen](#)
 - 独自の初期粒子発生方法を学ぶ
4. [P07_PrimaryGenerator/MyGen_MagField](#)
 - 一様磁場の設定方法を学ぶ



P07_PrimaryGenerator/GPSプログラムの概要

■ 演習プログラムの目的

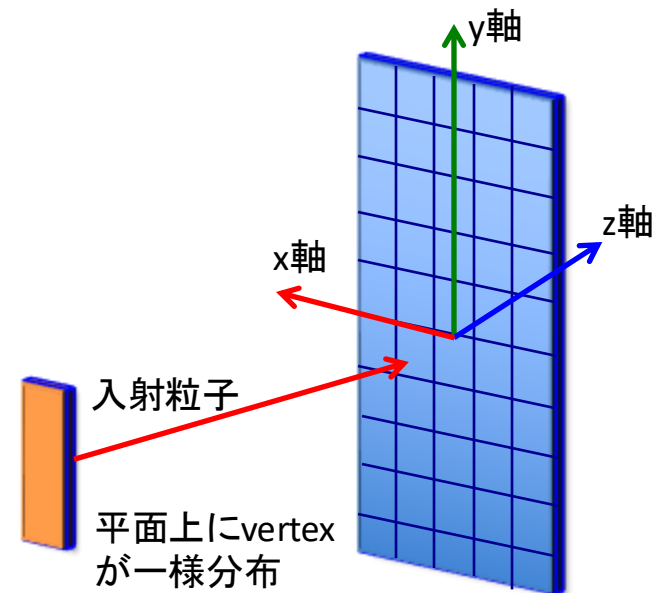
- 先の演習ではPrimary GeneratorとしてG4ParticleGunを使用
- 本演習ではツールキットが提供するもう一つのGeneratorであるG4GeneralParticleSource(GPS)のセットアップ方法を学ぶ
- 新たに作るプログラムはP07_PrimaryGenerator/GPSとよぶ
- GPSの基本的なUIコマンドの使用方法を学ぶ

■ プログラムの構成

- mainプログラム: 前演習と同一
- ジオメトリ: 前演習と同一
- PhysicsList: 前演習と同一
- PrimaryGenerator: 前演習のGunをGPSに変更

■ 組み込まれているジオメトリと粒子発生機能

- 粒子発生: GPSを使い平面に一様に分布するvertexから粒子をPixelに向けて照射



P07_PrimaryGenerator/GPS プログラムのビルド

課題:1 プログラム構成を理解する

[注] コマンド入力は「tabキー」補完機能を使う

- 前課題で使ったP07_PrimaryGenerator/Gunコードの以下の2つのファイルをGPS用に置換
 1. PrimaryGenerator.hh
 2. PrimaryGenerator.cc
- 上のコード以外は前演習と同一

課題:2 P07_PrimaryGenerator/GPS のソースコードをビルドして実行する

- 1) P07_PrimaryGenerator/GPSのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2025/P07_PrimaryGenerator/GPS
$ ls
source/  util/  TestBench/
```

- 2) ソースをビルド

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

← Scriptを走らせるとアプリ実行環境が完成

← ソースのビルドでbin、buildが作られた

- 3) TestBench(作業ディレクトリ)に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

P07_PrimaryGenerator/GPSプログラムを使う

課題:1 実行されたプログラムを使う

1) Qt画面のコマンド窓に以下を入力しアプリ実行

```
/control/execute GlobalSetup.mac
```

または、Qtのファイルアイコンをクリック

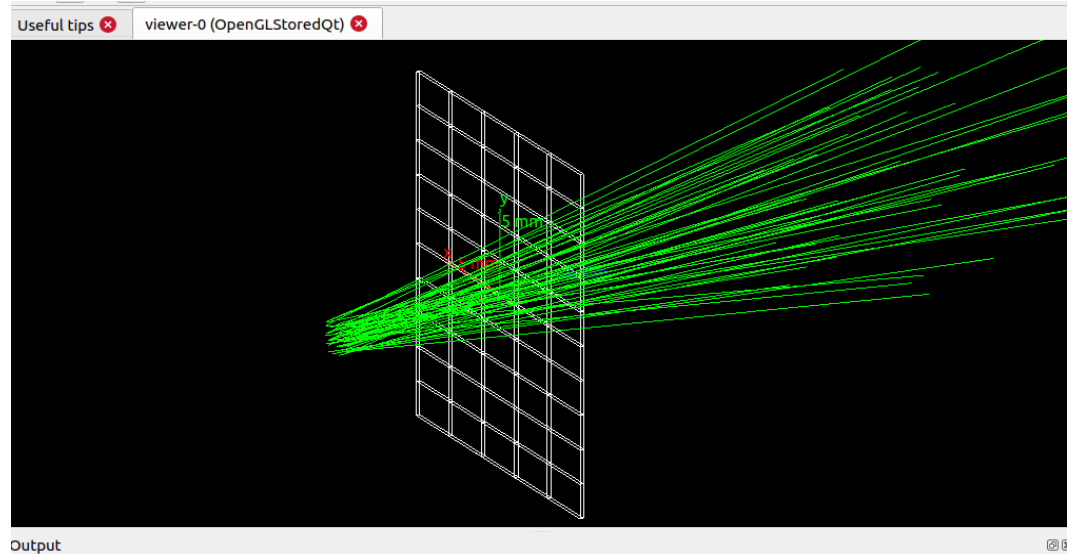
2) Qtコマンド窓に以下のコマンドを入力

```
/run/beamOn 50
```

- 右図の様に50本のgamma飛跡を確認する

[注] TestBenchのprimaryGeneratorSetup.macに
以下のように初期粒子設定がしてある

```
/gps/particle gamma  
/gps/pos/type Plane  
/gps/pos/shape Square  
/gps/pos/centre 0 0 -1 cm  
/gps/pos/halfx 1 mm  
/gps/pos/halfy 1 mm/gps/ang/type cos  
/gps/ang/maxtheta 10 deg  
/gps/ene/type Lin  
/gps/ene/min 2 MeV  
/gps/ene/max 10 MeV  
/gps/ene/gradient 1  
/gps/ene/intercept 1
```



GPSの機能は豊富であるが、ユーザ要求を全て満足させる一般性はない。
自由度の高い初期粒子発生が必要なら、次節の手法を用いることを推奨。

```
$ cd TestBench
```

```
$ less primaryGeneratorSetup.mac
```

上記の設定内容を確認する

3) アプリを終了

```
exit
```

次の課題では、実行したプログラムのGeometryクラスの
コードを読み、実装手法を学ぶ

P07_PrimaryGenerator/GPS: PrimaryGenerator.hhの内容

課題:1 GunのPrimaryGenerator.hhとGPSのPrimaryGenerator.hhをcolordiffコマンドで比較

```
$ cd ../source/include
$ colordiff -y --width=160 ../../../Gun/source/include/PrimaryGenerator.hh PrimaryGenerator.hh
```

[注] colordiffのversionで出力形式が異なる可能性がある

課題:2 colordiffの結果から変更された箇所を確認する

変更前ファイル		変更後ファイル	
<pre>//+++++ // PrimaryGenerator.hh //+++++ #ifndef PrimaryGenerator_h #define PrimaryGenerator_h 1 #include "G4VUserPrimaryGeneratorAction.hh" class G4Event; class G4ParticleGun; //----- class PrimaryGenerator : public G4VUserPrimaryGeneratorAction //----- { public: PrimaryGenerator(); ~PrimaryGenerator(); public: void GeneratePrimaries(G4Event*); private: G4ParticleGun* fpParticleGun; }; #endif</pre>		<pre>//+++++ // PrimaryGenerator.hh //+++++ #ifndef PrimaryGenerator_h #define PrimaryGenerator_h 1 #include "G4VUserPrimaryGeneratorAction.hh" class G4Event; class G4GeneralParticleSource; //----- class PrimaryGenerator : public G4VUserPrimaryGeneratorAction //----- { public: PrimaryGenerator(); ~PrimaryGenerator(); public: void GeneratePrimaries(G4Event*); private: G4GeneralParticleSource* fpParticleGPS; }; #endif</pre>	
	変更		変更

P07_PrimaryGenerator/GPS: PrimaryGenerator.ccの内容

課題:1 GunのPrimaryGenerator.ccとGPSのPrimaryGenerator.ccをcolordiffコマンドで比較

```
$ cd ../source/src
$ colordiff -y --width=160 ../../../Gun/source/src/PrimaryGenerator.cc PrimaryGenerator.cc
```

[注] colordiffのversionで出力形式が異なる可能性がある

課題:2 colordiffの結果から新たに追加された箇所を確認する

変更前ファイル		変更後ファイル	
//+++++ // PrimaryGenerator.cc //+++++		> //+++++ // PrimaryGenerator.cc //+++++	
#include "PrimaryGenerator.hh"		#include "PrimaryGenerator.hh"	
#include "G4ParticleGun.hh"	変更	#include "G4GeneralParticleSource.hh"	
----- PrimaryGenerator::PrimaryGenerator() : fpParticleGun{nullptr}		----- PrimaryGenerator::PrimaryGenerator() : fpParticleGPS{nullptr}	
----- { fpParticleGun = new G4ParticleGun{};		----- { fpParticleGPS = new G4GeneralParticleSource{};	
}		}	
----- PrimaryGenerator::~PrimaryGenerator() ----- { delete fpParticleGun;		----- PrimaryGenerator::~PrimaryGenerator() ----- { delete fpParticleGPS;	
}		}	
----- void PrimaryGenerator::GeneratePrimaries(G4Event* anEvent) ----- { fpParticleGun->GeneratePrimaryVertex(anEvent);		----- void PrimaryGenerator::GeneratePrimaries(G4Event* anEvent) ----- { fpParticleGPS->GeneratePrimaryVertex(anEvent);	
}		}	

P07_PrimaryGenerator/MyGen

独自の初期粒子発生方法を学ぶ

演習トピックス

[概要] [P07_PrimaryGenerator](#)プログラムを使い、初期粒子発生の設定方法を学ぶ

[目次]

1. [P07_PrimaryGenerator/Gun](#)
 - Particle Gunの設定方法
2. [P07_PrimaryGenerator/GPS](#)
 - General Particle Sourceの使い方
3. [P07_PrimaryGenerator/MyGen](#)
 - 独自の初期粒子発生方法を学ぶ
4. [P07_PrimaryGenerator/MyGen_MagField](#)
 - 一様磁場の設定方法を学ぶ



P07_PrimaryGenerator/MyGenプログラムの概要

■ 演習プログラムの目的

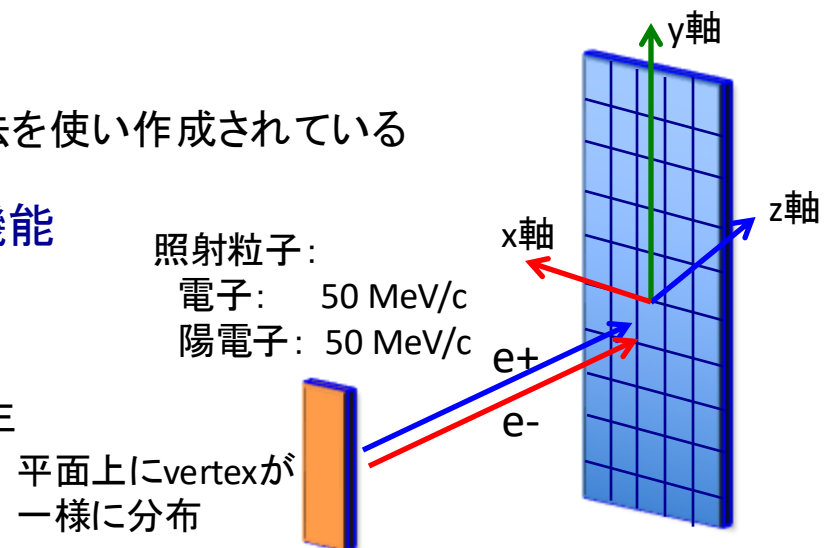
- G4ParticleGun及びG4GeneralParticleSourceは手軽に初期粒子を発生させるツール
- ここでは、これらのツールの基礎となっている、標準的なPrimary Generator作成手法を学ぶ
 ← Gun/GPSを使用せずに、ユーザが簡単に自由度の高い粒子発生ができる
- まずは単純に電子・陽電子を一つずつ発生させるgeneratorを作り、作成の基礎を学ぶ
 ← コマンド処理機能はないが、バッチ処理用のアプリを作成するにはこれで十分

■ プログラムの構成

- mainプログラム: 前演習と同一
- ジオメトリ: 前演習と同一
- PhysicsList: 前演習と同一
- PrimaryGenerator: 一般的な粒子発生手法を使い作成されている

■ 組み込まれているジオメトリと粒子発生機能

- ジオメトリ: Pixel_One検出器
- 粒子発生: 平面上からランダムに
 電子と陽電子を1個ずつ発生



P07_PrimaryGenerator/MyGen プログラムのビルド

[注] コマンド入力は「tabキー」補完機能を使う

課題:1 プログラム構成を理解する

- 前課題で使ったP07_PrimaryGenerator/GPSの以下の2つのファイルをMyGen用に置換
 1. PrimaryGenerator.hh
 2. PrimaryGenerator.cc
- 上のコード以外は前演習と同一

課題:2 P07_PrimaryGenerator/MyGen のソースコードをビルドして実行する

- 1) P07_PrimaryGenerator/MyGenのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2025/P07_PrimaryGenerator/MyGen
$ ls
source/  util/  TestBench/
```

- 2) ソースをビルド

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

← Scriptを走らせるとアプリ実行環境が完成

← ソースのビルドでbin、buildが作られた

- 3) TestBench(作業ディレクトリ) に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

P07_PrimaryGenerator/MyGenプログラムを使う

課題:1 実行されたプログラムを使う

1) Qt画面のコマンド窓に以下を入力しアプリ実行

```
/control/execute GlobalSetup.mac
```

または、Qtのファイルアイコンをクリック

2) Qtコマンド窓に以下のコマンドを入力

```
/run/beamOn 20
```

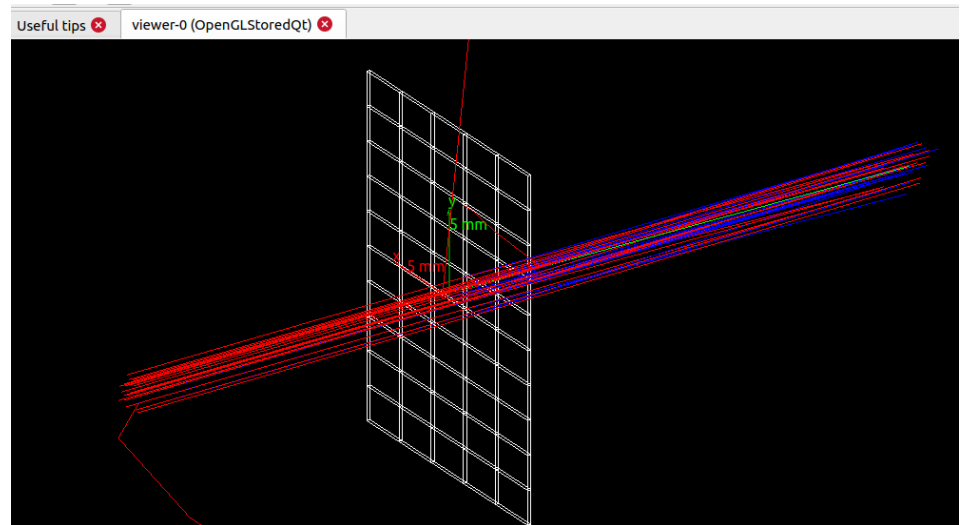
- 右図の様に総計20本のelectronとpositron飛跡を確認する

[注]

MyGenのPrimaryGeneratorはコントロールコマンドが実装されていない。発生粒子は'/run/beamOn 1'でelectronとpositronのみ。enefgy, position等も固定されている。

3) アプリを終了

```
exit
```



次の課題では、実行したプログラムのPrimaryGeneratorクラスのコードを読み、実装手法を学ぶ

P07_PrimaryGenerator/MyGen: PrimaryGenerator.hhの内容

課題:1 GPSのPrimaryGenerator.hhとMyGenのPrimaryGenerator.hhをcolordiffコマンドで比較

```
$ cd ../source/include
$ colordiff -y --width=160 ../../GPS/source/include/PrimaryGenerator.hh PrimaryGenerator.hh
```

[注] colordiffのversionで出力形式が異なる可能性がある

課題:2 colordiffの結果から変更された箇所を確認する

GPSファイル	MyGenファイル
++++++ // PrimaryGenerator.hh ++++++ #ifndef PrimaryGenerator_h #define PrimaryGenerator_h 1 #include "G4VUserPrimaryGeneratorAction.hh" class G4Event; class G4GeneralParticleSource; ----- class PrimaryGenerator : public G4VUserPrimaryGeneratorAction ----- { public: PrimaryGenerator(); ~PrimaryGenerator() override; public: void GeneratePrimaries(G4Event*) override; private: G4GeneralParticleSource* fpParticleGPS; }; #endif	++++++ // PrimaryGenerator.hh ++++++ #ifndef PrimaryGenerator_h #define PrimaryGenerator_h 1 #include "G4VUserPrimaryGeneratorAction.hh" > #include "G4ThreeVector.hh" > class G4ParticleDefinition; class G4Event; < ----- class PrimaryGenerator : public G4VUserPrimaryGeneratorAction ----- { public: PrimaryGenerator(); ~PrimaryGenerator() override; public: void GeneratePrimaries(G4Event*) override; < < < }; #endif

この関数をユーザが実装すれば、独自の粒子発生機構作れる

← 次のスライド参照

P07_PrimaryGenerator/MyGen: PrimaryGenerator.ccの内容

課題:3 MyGenの実装ファイルを理解する

```
$ less ../source/src/PrimaryGenerator.cc
```

```
//+++++
// PrimaryGenerator.cc
//+++++
#include "PrimaryGenerator.hh"
#include "G4Event.hh"
#include "G4PrimaryVertex.hh"
#include "G4PrimaryParticle.hh"
#include "G4ParticleTable.hh"
#include "G4SystemOfUnits.hh"
#include "Randomize.hh"
```

== Codeの不変部分を省略 ==

```
//-----
void PrimaryGenerator::GeneratePrimaries( G4Event* anEvent )
//-----
{
  // Particle table
  G4ParticleTable* particleTable = G4ParticleTable::GetParticleTable();

  // Create a primary particle - need to create for every event
  G4String particleName = "e-";
  G4double momentum = 50.0*MeV;
  G4ThreeVector momentumDirection = G4ThreeVector{ 0.0, 0.0, 1.0 };
  G4ThreeVector momVect = momentumDirection*momentum;
  auto primaryParticle =
    new G4PrimaryParticle{ particleTable->FindParticle( particleName ),
                          momVect.x(), momVect.y(), momVect.z() };

  // Create a 2nd primary particle - need to create for every event
  particleName = "e+";
  auto primaryParticle_2nd =
    new G4PrimaryParticle{ particleTable->FindParticle( particleName ),
                          momVect.x(), momVect.y(), momVect.z() };

  // Create a primary vertex - need to create for every event
  G4double pos_X = 2.0*mm*( G4UniformRand()-0.5 );
  G4double pos_Y = 2.0*mm*( G4UniformRand()-0.5 );
  G4double pos_Z = -2.0*cm;
  auto vertex = G4ThreeVector{ pos_X, pos_Y, pos_Z };
  G4double time_Zero = 0.0*ns;
  auto primaryVertex = new G4PrimaryVertex{ vertex, time_Zero };

  // Add the primary particles to the primary vertex
  primaryVertex->SetPrimary( primaryParticle );
  primaryVertex->SetPrimary( primaryParticle_2nd );

  // Add the vertex to the event
  anEvent->AddPrimaryVertex( primaryVertex );
}
```

GeneratePrimariesメソッドを
以下の二つのクラスを使い実装

- G4PrimaryParticle
- G4PrimaryVertex

[注] これらのクラスについては
「初期粒子の発生方法」講義スライド
参照

particle tableへのポインタを取得

発生させるelectronの物理情報の設定

G4PrimaryParticleクラスからnewで
electronを生成 (事象発生ごとに毎回new)

発生させるpositronの物理情報は
electronと同一

G4PrimaryParticleクラスからnewで
positronを生成 (事象発生ごとに毎回new)

G4PrimaryVertexクラスからnewで
primary vertexを生成
(事象発生ごとに毎回new)

electron/positronをprimary vertex
に配置

primary vertexをG4Eventクラスへの
ポインター(anEvent)に付加

P07_PrimaryGenerator/MyGen_MagField

一様磁場の設定方法を学ぶ

演習トピックス

[概要] [P07_PrimaryGenerator](#)プログラムを使い、初期粒子発生の設定方法を学ぶ

[目次]

1. [P07_PrimaryGenerator/Gun](#)
 - Particle Gunの設定方法
2. [P07_PrimaryGenerator/GPS](#)
 - General Particle Sourceの使い方
3. [P07_PrimaryGenerator/MyGen](#)
 - 独自の初期粒子発生方法を学ぶ
4. [P07_PrimaryGenerator/MyGen_MagField](#)
 - 一様磁場の設定方法を学ぶ



P07_PrimaryGenerator/MyGen_MagFieldプログラムの概要

■ 演習プログラムの目的

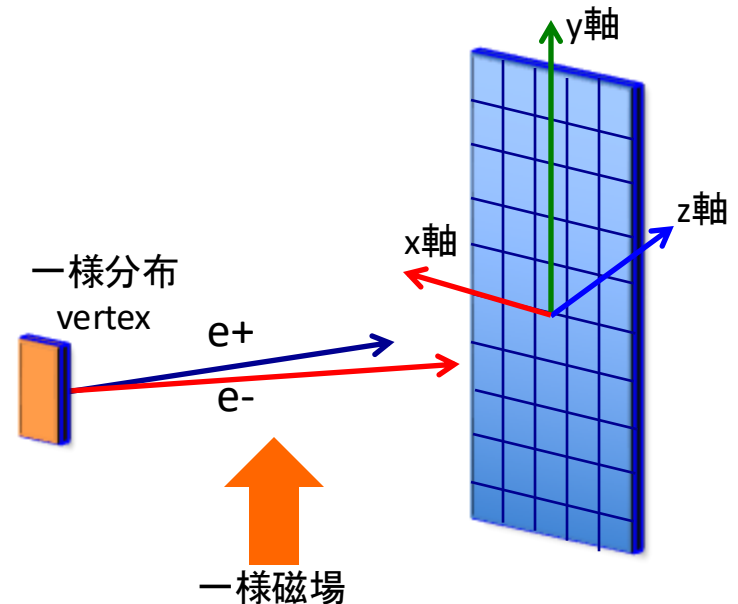
- 前演習のMyGenで使用したGeometryを使い、一様磁場を全空間に設定する方法を学ぶ

■ プログラムの構成

- mainプログラム: 前演習と同一
- ジオメトリ: 前演習のジオメトリ(Pixel_One検出器)に**一様磁場を追加する**
- PhysicsList: 前演習と同一
- PrimaryGenerator: 前演習と同一(MyGenコード)

■ 組み込まれているジオメトリと粒子発生機能

- ジオメトリ: Pixel_One検出器と一様磁場
- 粒子発生: 平面上に一様分布するvertexから同時に電子と陽電子を発生



P07_PrimaryGenerator/MyGen_MagField プログラムのビルド

課題:1 プログラム構成を理解する

[注] コマンド入力は「tabキー」補完機能を使う

- 前課題のジオメトリ(Geometry.cc)に一樣磁場が追加されている
- 上のコード以外は前演習と同一

課題:2 P07_PrimaryGenerator/MyGen_MagField のソースコードをビルドして実行する

1) P07_PrimaryGenerator/MyGen_MagFieldのディレクトリに移行する

```
$ cd ~/Geant4Tutorial_2025/07_PrimaryGenerator/MyGen_MagField
$ ls
source/  util/  TestBench/
```

2) ソースをビルド

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

← Scriptを走らせるとアプリ実行環境が完成

← ソースのビルドでbin、buildが作られた

3) TestBench(作業ディレクトリ) に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

P07_PrimaryGenerator/MyGen_MagFieldプログラムを使う

課題:1 実行されたプログラムを使う

1) Qt画面のコマンド窓に以下を入力しアプリ実行

```
/control/execute GlobalSetup.mac
```

または、Qtのファイルアイコンをクリック

2) Qtコマンド窓に以下のコマンドを入力

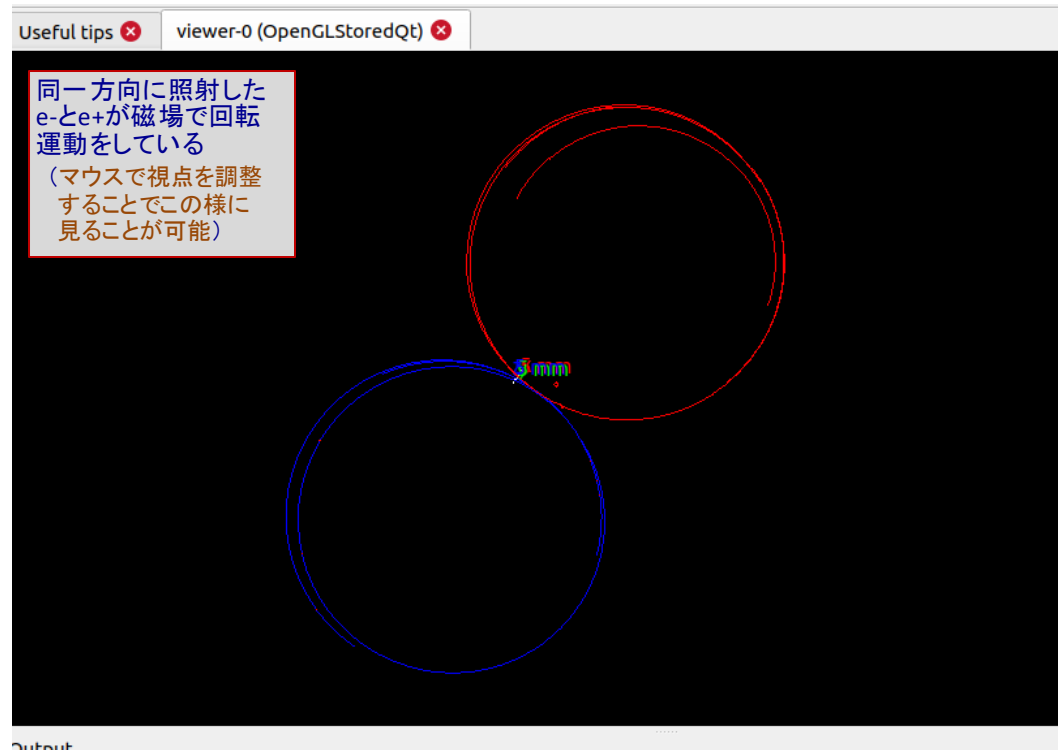
```
/run/beamOn 1
```

- 右図の様に

3) アプリを終了

```
exit
```

次の課題では、実行したプログラムの
Geometryクラスのコードを読み、実装手
法を学ぶ



演習 終了